User

Speichert alle registrierten Benutzenden der Plattform inklusive grundlegender Profil- und Kontaktdaten.

Zur Erstellung der Tabelle:

```
CREATE TABLE `user` (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(120),
    email VARCHAR(180) UNIQUE,
    phone VARCHAR(30),
    zip VARCHAR(12),
    city VARCHAR(80),
    created_at DATETIME
);
```

Mit dieser Abfrage wird geprüft, ob Benutzende korrekt gespeichert und nach Standort gefiltert werden können.

```
SELECT user_id, full_name, email, city
FROM `user`
WHERE city = 'Karlsruhe'
ORDER BY full_name;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `user` (full_name, email, phone, zip, city, created_at) VALUES
('Max Mustermann','max@mail.de','123456','76133','Karlsruhe',NOW()),
('Anna Schmidt','anna@mail.de','111111','76131','Karlsruhe',NOW()),
('Tom Becker','tom@mail.de','222222','76135','Karlsruhe',NOW()),
('Lisa Weber','lisa@mail.de','333333','76137','Karlsruhe',NOW()),
('Paul Fischer','paul@mail.de','444444','76139','Karlsruhe',NOW()),
('Sara Klein','sara@mail.de','555555','76149','Karlsruhe',NOW()),
('Jan Wolf','jan@mail.de','666666','76185','Karlsruhe',NOW()),
('Mia Koch','mia@mail.de','777777','76187','Karlsruhe',NOW()),
('Tim Braun','tim@mail.de','888888','76189','Karlsruhe',NOW()),
('Lea Mayer','lea@mail.de','999999','76199','Karlsruhe',NOW());
```

Ergebnis der Abfrage:

←T→				user_id	full_name 1	email	city
☐	Bearbeiten	Kopieren	Löschen	2	Anna Schmidt	anna@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	7	Jan Wolf	jan@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	10	Lea Mayer	lea@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	4	Lisa Weber	lisa@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	1	Max Mustermann	max@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	8	Mia Koch	mia@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	5	Paul Fischer	paul@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	6	Sara Klein	sara@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	9	Tim Braun	tim@mail.de	Karlsruhe
☐	Bearbeiten	Kopieren	Löschen	3	Tom Becker	tom@mail.de	Karlsruhe

Role

Definiert die verschiedenen Rollen im System, z. B. Gast, registrierter Benutzer oder Administrator.

Zur Erstellung der Tabelle:

```
CREATE TABLE `role` (
   role_id INT PRIMARY KEY AUTO_INCREMENT,
   role_name VARCHAR(40)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `role` (role_name) VALUES
('GAST'),('USER'),('ADMIN'),
('MODERATOR'),('TESTrole1'),('TESTrole2'),
('TESTrole3'),('TESTrole4'),('TESTrole5'),('TESTrole6');
```

Mit dieser Abfrage wird geprüft, ob Rollen korrekt angelegt und sortiert ausgegeben werden.

```
SELECT role_id, role_name
FROM `role`
ORDER BY role_name;
```

Ergebnis der Abfrage:

←T→				role_id	role_name 1
☐	Bearbeiten	Kopieren	Löschen	3	ADMIN
☐	Bearbeiten	Kopieren	Löschen	1	GAST
☐	Bearbeiten	Kopieren	Löschen	4	MODERATOR
☐	Bearbeiten	Kopieren	Löschen	5	TESTrole1
☐	Bearbeiten	Kopieren	Löschen	6	TESTrole2
☐	Bearbeiten	Kopieren	Löschen	7	TESTrole3
☐	Bearbeiten	Kopieren	Löschen	8	TESTrole4
☐	Bearbeiten	Kopieren	Löschen	9	TESTrole5
☐	Bearbeiten	Kopieren	Löschen	10	TESTrole6
☐	Bearbeiten	Kopieren	Löschen	2	USER

UserRole

Verknüpft Benutzer mit ihren Rollen und ermöglicht eine flexible Rollenvergabe (n:m-Beziehung).

Zur Erstellung der Tabelle:

```
CREATE TABLE `userrole` (
   user_id INT,
   role_id INT,
   PRIMARY KEY (user_id, role_id),
   FOREIGN KEY (user_id) REFERENCES `user`(user_id),
   FOREIGN KEY (role_id) REFERENCES `role`(role_id)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `userrole` (user_id, role_id) VALUES
(1,2),(2,2),(3,2),(4,2),(5,2),
(6,2),(7,2),(8,2),(9,2),(10,2);
```

Mit dieser Abfrage wird geprüft, ob die Zuordnung zwischen Benutzenden und Rollen korrekt funktioniert.

```
SELECT u.full_name, r.role_name
FROM `userrole` ur
JOIN `user` u ON ur.user_id = u.user_id
JOIN `role` r ON ur.role_id = r.role_id
ORDER BY u.full_name;
```

Ergebnis der Abfrage:

full_name ▲ 1	role_name
Anna Schmidt	USER
Jan Wolf	USER
Lea Mayer	USER
Lisa Weber	USER
Max Mustermann	USER
Mia Koch	USER
Paul Fischer	USER
Sara Klein	USER
Tim Braun	USER
Tom Becker	USER

Publisher

Speichert Informationen zu Verlagen, die Bücher veröffentlichen.

Zur Erstellung der Tabelle:

```
CREATE TABLE `publisher` (
    publisher_id INT PRIMARY KEY AUTO_INCREMENT,
    publisher_name VARCHAR(120)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `publisher` (publisher_name) VALUES
('Verlag A'),('Verlag B'),('Verlag C'),
('Verlag D'),('Verlag E'),('Verlag F'),
('Verlag G'),('Verlag H'),('Verlag I'),('Verlag J');
```

Mit dieser Abfrage wird geprüft, ob Verlage korrekt gespeichert sind und wie viele Bücher ihnen zugeordnet sind.

```
SELECT p.publisher_name, COUNT(b.book_id) AS anzahl_buecher
FROM `publisher` p
LEFT JOIN `book` b ON p.publisher_id = b.publisher_id
GROUP BY p.publisher_id, p.publisher_name
ORDER BY anzahl_buecher DESC, p.publisher_name;
```

Ergebnis der Abfrage:

publisher_name	anzahl_buecher 1
Verlag A	1
Verlag B	1
Verlag C	1
Verlag D	1
Verlag E	1
Verlag F	1
Verlag G	1
Verlag H	1
Verlag I	1
Verlag J	1

Book

Enthält bibliografische Informationen zu Büchern wie Titel, Sprache, Erscheinungsjahr und ISBN.

Zur Erstellung der Tabelle:

```
CREATE TABLE `book` (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200),
    language VARCHAR(40),
    publication_year SMALLINT,
    isbn VARCHAR(20) UNIQUE,
    publisher_id INT,
    FOREIGN KEY (publisher_id) REFERENCES `publisher`(publisher_id)
);
```

Mit dieser Abfrage wird geprüft, ob Buchdaten korrekt gespeichert und mit Verlagen verknüpft sind.

```
SELECT b.title, b.language, b.publication_year, p.publisher_name
FROM `book` b
JOIN `publisher` p ON b.publisher_id = p.publisher_id
ORDER BY b.publication_year DESC, b.title;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `book` (title, language, publication_year, isbn, publisher_id) VALUES
('Buch 1','DE',2020,'ISBN1',1),
('Buch 2','DE',2021,'ISBN2',2),
('Buch 3','EN',2019,'ISBN3',3),
('Buch 4','DE',2018,'ISBN4',4),
('Buch 5','EN',2022,'ISBN5',5),
('Buch 6','DE',2017,'ISBN6',6),
('Buch 7','EN',2016,'ISBN7',7),
('Buch 8','DE',2015,'ISBN8',8),
('Buch 9','EN',2014,'ISBN9',9),
('Buch 10','DE',2013,'ISBN10',10);
```

Ergebnis der Abfrage:

title	language	publication_year ▼ 1	publisher_name
Buch 5	EN	2022	Verlag E
Buch 2	DE	2021	Verlag B
Buch 1	DE	2020	Verlag A
Buch 3	EN	2019	Verlag C
Buch 4	DE	2018	Verlag D
Buch 6	DE	2017	Verlag F
Buch 7	EN	2016	Verlag G
Buch 8	DE	2015	Verlag H
Buch 9	EN	2014	Verlag I
Buch 10	DE	2013	Verlag J

Author

Speichert die Autoren von Büchern und ermöglicht die Zuordnung zu mehreren Werken.

Zur Erstellung der Tabelle:

```
CREATE TABLE `author` (
   author_id INT PRIMARY KEY AUTO_INCREMENT,
   author_name VARCHAR(120)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `author` (author_name) VALUES
('Autor 1'),('Autor 2'),('Autor 3'),
('Autor 4'),('Autor 5'),('Autor 6'),
('Autor 7'),('Autor 8'),('Autor 9'),('Autor 10');
```

Mit dieser Abfrage wird geprüft, ob Autoren korrekt gespeichert sind und ihre Buchzuordnungen funktionieren.

```
SELECT a.author_name, COUNT(ba.book_id) AS anzahl_buecher
FROM `author` a
LEFT JOIN `bookauthor` ba ON a.author_id = ba.author_id
GROUP BY a.author_id, a.author_name
ORDER BY anzahl_buecher DESC, a.author_name;
```

Ergebnis der Abfrage:

author_name	anzahl_buecher 1
Autor 1	1
Autor 10	1
Autor 2	1
Autor 3	1
Autor 4	1
Autor 5	1
Autor 6	1
Autor 7	1
Autor 8	1
Autor 9	1

BookAuthor

Verknüpft Bücher mit ihren Autoren und bildet die n:m-Beziehung zwischen beiden Entitäten ab.

Zur Erstellung der Tabelle:

```
CREATE TABLE `bookauthor` (
   book_id INT,
   author_id INT,
   PRIMARY KEY (book_id, author_id),
   FOREIGN KEY (book_id) REFERENCES `book`(book_id),
   FOREIGN KEY (author_id) REFERENCES `author`(author_id)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `bookauthor` (book_id, author_id) VALUES
(1,1),(2,2),(3,3),(4,4),(5,5),
(6,6),(7,7),(8,8),(9,9),(10,10);
```

Mit dieser Abfrage wird geprüft, ob die n:m-Beziehung zwischen Büchern und Autoren korrekt umgesetzt ist.

```
SELECT b.title, a.author_name
FROM `bookauthor` ba
JOIN `book` b ON ba.book_id = b.book_id
JOIN `author` a ON ba.author_id = a.author_id
ORDER BY b.title;
```

Ergebnis der Abfrage:

title ▲ 1	author_name
Buch 1	Autor 1
Buch 10	Autor 10
Buch 2	Autor 2
Buch 3	Autor 3
Buch 4	Autor 4
Buch 5	Autor 5
Buch 6	Autor 6
Buch 7	Autor 7
Buch 8	Autor 8
Buch 9	Autor 9

Genre

Definiert verschiedene Buchgenres zur Kategorisierung von Titeln.

Zur Erstellung der Tabelle:

```
CREATE TABLE `genre` (
   genre_id INT PRIMARY KEY AUTO_INCREMENT,
   genre_name VARCHAR(80)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `genre` (genre_name) VALUES
('Roman'),('Thriller'),('Fantasy'),
('Sci-Fi'),('Krimi'),('Sachbuch'),
('Biografie'),('Drama'),('Horror'),('Kinderbuch');
```

Mit dieser Abfrage wird geprüft, ob Genres korrekt verwaltet und Büchern zugeordnet werden.

```
SELECT g.genre_name, COUNT(bg.book_id) AS anzahl_buecher
FROM `genre` g
LEFT JOIN `bookgenre` bg ON g.genre_id = bg.genre_id
GROUP BY g.genre_id, g.genre_name
ORDER BY anzahl_buecher DESC, g.genre_name;
```

Ergebnis der Abfrage:

genre_name	anzahl_buecher 1
Biografie	1
Drama	1
Fantasy	1
Horror	1
Kinderbuch	1
Krimi	1
Roman	1
Sachbuch	1
Sci-Fi	1
Thriller	1

BookGenre

Ordnet Bücher einem oder mehreren Genres zu und realisiert die n:m-Beziehung.

Zur Erstellung der Tabelle:

```
CREATE TABLE `bookgenre` (
   book_id INT,
   genre_id INT,
   PRIMARY KEY (book_id, genre_id),
   FOREIGN KEY (book_id) REFERENCES `book`(book_id),
   FOREIGN KEY (genre_id) REFERENCES `genre`(genre_id)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `bookgenre` (book_id, genre_id) VALUES
(1,1),(2,2),(3,3),(4,4),(5,5),
(6,6),(7,7),(8,8),(9,9),(10,10);
```

Mit dieser Abfrage wird geprüft, ob die n:m-Beziehung zwischen Büchern und Genres korrekt funktioniert.

```
SELECT b.title, g.genre_name
FROM `bookgenre` bg
JOIN `book` b ON bg.book_id = b.book_id
JOIN `genre` g ON bg.genre_id = g.genre_id
ORDER BY b.title;
```

Ergebnis der Abfrage:

title ▲ 1	genre_name
Buch 1	Roman
Buch 10	Kinderbuch
Buch 2	Thriller
Buch 3	Fantasy
Buch 4	Sci-Fi
Buch 5	Krimi
Buch 6	Sachbuch
Buch 7	Biografie
Buch 8	Drama
Buch 9	Horror

DeliveryMethod

Definiert die möglichen Lieferarten, z. B. Abholung oder Versand.

Zur Erstellung der Tabelle:

```
CREATE TABLE `deliverymethod` (
    delivery_method_id INT PRIMARY KEY AUTO_INCREMENT,
    method_name VARCHAR(30)
);
```

Mit dieser Abfrage wird geprüft, ob Liefermethoden korrekt gespeichert und abrufbar sind.

```
SELECT delivery_method_id, method_name
FROM `deliverymethod`
ORDER BY method_name;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `deliverymethod` (method_name) VALUES
('PICKUP'),('SHIPPING'),('TESTdelivery1'),('TESTdelivery2'),
('TESTdelivery3'),('TESTdelivery4'),('TESTdelivery5'),('TESTdelivery6'),
('TESTdelivery7'),('TESTdelivery8');
```

Ergebnis der Abfrage:

←T→				delivery_method_id	method_name ▲ 1
☐	Bearbeiten	Kopieren	Löschen	1	PICKUP
☐	Bearbeiten	Kopieren	Löschen	2	SHIPPING
☐	Bearbeiten	Kopieren	Löschen	3	TESTdelivery1
☐	Bearbeiten	Kopieren	Löschen	4	TESTdelivery2
☐	Bearbeiten	Kopieren	Löschen	5	TESTdelivery3
☐	Bearbeiten	Kopieren	Löschen	6	TESTdelivery4
☐	Bearbeiten	Kopieren	Löschen	7	TESTdelivery5
☐	Bearbeiten	Kopieren	Löschen	8	TESTdelivery6
☐	Bearbeiten	Kopieren	Löschen	9	TESTdelivery7
☐	Bearbeiten	Kopieren	Löschen	10	TESTdelivery8

BookCopy

Repräsentiert konkrete Exemplare eines Buches mit individuellen Eigenschaften wie Zustand und Verfügbarkeit.

Zur Erstellung der Tabelle:

```
CREATE TABLE `bookcopy` (
  copy_id INT PRIMARY KEY AUTO_INCREMENT,
  book_id INT,
  owner_user_id INT,
  condition_grade VARCHAR(20),
  max_loan_days INT,
  shipping_allowed BOOLEAN,
  deposit_amount DECIMAL(8,2),
  status VARCHAR(20),
  FOREIGN KEY (book_id) REFERENCES `book`(book_id),
  FOREIGN KEY (owner_user_id) REFERENCES `user`(user_id)
);
```

Mit dieser Abfrage wird geprüft, ob Exemplare korrekt einem Buch und einem Besitzer zugeordnet sind.

```
SELECT bc.copy_id, b.title, u.full_name AS owner, bc.condition_grade, bc.status
FROM `bookcopy` bc
JOIN `book` b ON bc.book_id = b.book_id
JOIN `user` u ON bc.owner_user_id = u.user_id
ORDER BY bc.copy_id;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `bookcopy` (book_id, owner_user_id, condition_grade, max_loan_days, shipping_allowed, deposit_amount, status) VALUES
(1,1,'gut',14,1,5.00,'verfügbar'),
(2,2,'neu',10,1,3.00,'verfügbar'),
(3,3,'mittel',7,0,2.00,'verfügbar'),
(4,4,'gut',14,1,4.00,'verfügbar'),
(5,5,'alt',5,0,1.00,'verfügbar'),
(6,6,'gut',10,1,2.00,'verfügbar'),
(7,7,'neu',12,1,3.00,'verfügbar'),
(8,8,'mittel',8,0,2.00,'verfügbar'),
(9,9,'gut',15,1,5.00,'verfügbar'),
(10,10,'neu',20,1,6.00,'verfügbar');
```

Ergebnis der Abfrage:

copy_id ▲ 1	title	owner	condition_grade	status
1	Buch 1	Max Mustermann	gut	verfügbar
2	Buch 2	Anna Schmidt	neu	verfügbar
3	Buch 3	Tom Becker	mittel	verfügbar
4	Buch 4	Lisa Weber	gut	verfügbar
5	Buch 5	Paul Fischer	alt	verfügbar
6	Buch 6	Sara Klein	gut	verfügbar
7	Buch 7	Jan Wolf	neu	verfügbar
8	Buch 8	Mia Koch	mittel	verfügbar
9	Buch 9	Tim Braun	gut	verfügbar
10	Buch 10	Lea Mayer	neu	verfügbar

PickupLocation

Enthält Abholorte mit Adress- und Geokoordinaten für die Übergabe von Büchern.

Zur Erstellung der Tabelle:

```
CREATE TABLE `pickuplocation` (
  location_id INT PRIMARY KEY AUTO_INCREMENT,
  owner_user_id INT,
  street VARCHAR(120),
  zip VARCHAR(12),
  city VARCHAR(80),
  latitude DECIMAL(9,6),
  longitude DECIMAL(9,6),
  FOREIGN KEY (owner_user_id) REFERENCES `user`(user_id)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `pickuplocation` (owner_user_id, street, zip, city, latitude, longitude) VALUES
(1,'Straße 1','76133','Karlsruhe',49.0,8.4),
(2,'Straße 2','76131','Karlsruhe',49.1,8.5),
(3,'Straße 3','76135','Karlsruhe',49.2,8.6),
(4,'Straße 4','76137','Karlsruhe',49.3,8.7),
(5,'Straße 5','76139','Karlsruhe',49.4,8.8),
(6,'Straße 6','76149','Karlsruhe',49.5,8.9),
(7,'Straße 7','76185','Karlsruhe',49.6,8.1),
(8,'Straße 8','76187','Karlsruhe',49.7,8.2),
(9,'Straße 9','76189','Karlsruhe',49.8,8.3),
(10,'Straße 10','76199','Karlsruhe',49.9,8.0);
```

Mit dieser Abfrage wird geprüft, ob Abholorte korrekt gespeichert und Benutzenden zugeordnet sind.

```
SELECT pl.location_id, u.full_name AS owner, pl.street, pl.zip, pl.city
FROM `pickuplocation` pl
JOIN `user` u ON pl.owner_user_id = u.user_id
ORDER BY pl.city, pl.street;
```

Ergebnis der Abfrage:

location_id	owner	street	zip	city ▲ 1
1	Max Mustermann	Straße 1	76133	Karlsruhe
10	Lea Mayer	Straße 10	76199	Karlsruhe
2	Anna Schmidt	Straße 2	76131	Karlsruhe
3	Tom Becker	Straße 3	76135	Karlsruhe
4	Lisa Weber	Straße 4	76137	Karlsruhe
5	Paul Fischer	Straße 5	76139	Karlsruhe
6	Sara Klein	Straße 6	76149	Karlsruhe
7	Jan Wolf	Straße 7	76185	Karlsruhe
8	Mia Koch	Straße 8	76187	Karlsruhe
9	Tim Braun	Straße 9	76189	Karlsruhe

TimeSlot

Definiert Zeitfenster, in denen eine Übergabe eines Buches möglich ist.

Zur Erstellung der Tabelle:

```
CREATE TABLE `timeslot` (
   slot_id INT PRIMARY KEY AUTO_INCREMENT,
   weekday TINYINT,
   start_time TIME,
   end_time TIME
);
```

Mit dieser Abfrage wird geprüft, ob Zeitfenster korrekt gespeichert und sortiert dargestellt werden.

```
SELECT slot_id, weekday, start_time, end_time
FROM `timeslot`
ORDER BY weekday, start_time;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `timeslot` (weekday, start_time, end_time) VALUES
(1,'09:00:00','11:00:00'),
(2,'10:00:00','12:00:00'),
(3,'11:00:00','13:00:00'),
(4,'12:00:00','14:00:00'),
(5,'13:00:00','15:00:00'),
(6,'14:00:00','16:00:00'),
(7,'15:00:00','17:00:00'),
(1,'16:00:00','18:00:00'),
(2,'17:00:00','19:00:00'),
(3,'18:00:00','20:00:00');
```

Ergebnis der Abfrage:

←T→				slot_id	weekday ▲ 1	start_time ▲ 2	end_time
☐	Bearbeiten	Kopieren	Löschen	1	1	09:00:00	11:00:00
☐	Bearbeiten	Kopieren	Löschen	8	1	16:00:00	18:00:00
☐	Bearbeiten	Kopieren	Löschen	2	2	10:00:00	12:00:00
☐	Bearbeiten	Kopieren	Löschen	9	2	17:00:00	19:00:00
☐	Bearbeiten	Kopieren	Löschen	3	3	11:00:00	13:00:00
☐	Bearbeiten	Kopieren	Löschen	10	3	18:00:00	20:00:00
☐	Bearbeiten	Kopieren	Löschen	4	4	12:00:00	14:00:00
☐	Bearbeiten	Kopieren	Löschen	5	5	13:00:00	15:00:00
☐	Bearbeiten	Kopieren	Löschen	6	6	14:00:00	16:00:00
☐	Bearbeiten	Kopieren	Löschen	7	7	15:00:00	17:00:00

CopyAvailability

Verknüpft Exemplar, Abholort und Zeitfenster und beschreibt die konkrete Verfügbarkeit eines Buches.

Zur Erstellung der Tabelle:

```
CREATE TABLE `copyavailability` (
  copy_id INT,
  location_id INT,
  slot_id INT,
  PRIMARY KEY (copy_id, location_id, slot_id),
  FOREIGN KEY (copy_id) REFERENCES `bookcopy`(copy_id),
  FOREIGN KEY (location_id) REFERENCES `pickuplocation`(location_id),
  FOREIGN KEY (slot_id) REFERENCES `timeslot`(slot_id)
);
```

Zum Befüllen der Tabelle:

```
INSERT INTO `copyavailability` (copy_id, location_id, slot_id) VALUES
(1,1,1),(2,2,2),(3,3,3),(4,4,4),(5,5,5),
(6,6,6),(7,7,7),(8,8,8),(9,9,9),(10,10,10);
```

Mit dieser Abfrage wird geprüft, ob die Verfügbarkeit eines Exemplars über Ort und Zeit korrekt abgebildet wird.

```
SELECT bc.copy_id, b.title, pl.city, ts.weekday, ts.start_time, ts.end_time
FROM `copyavailability` ca
JOIN `bookcopy` bc ON ca.copy_id = bc.copy_id
JOIN `book` b ON bc.book_id = b.book_id
JOIN `pickuplocation` pl ON ca.location_id = pl.location_id
JOIN `timeslot` ts ON ca.slot_id = ts.slot_id
ORDER BY bc.copy_id, ts.weekday;
```

Ergebnis der Abfrage:

copy_id ▲ 1	title	city	weekday	start_time	end_time
1	Buch 1	Karlsruhe	1	09:00:00	11:00:00
2	Buch 2	Karlsruhe	2	10:00:00	12:00:00
3	Buch 3	Karlsruhe	3	11:00:00	13:00:00
4	Buch 4	Karlsruhe	4	12:00:00	14:00:00
5	Buch 5	Karlsruhe	5	13:00:00	15:00:00
6	Buch 6	Karlsruhe	6	14:00:00	16:00:00
7	Buch 7	Karlsruhe	7	15:00:00	17:00:00
8	Buch 8	Karlsruhe	1	16:00:00	18:00:00
9	Buch 9	Karlsruhe	2	17:00:00	19:00:00
10	Buch 10	Karlsruhe	3	18:00:00	20:00:00

Reservation

Speichert Anfragen von Benutzenden zur Ausleihe eines Buches inklusive Status und Liefermethode.

Zur Erstellung der Tabelle:

```
CREATE TABLE `reservation` (
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    borrower_user_id INT,
    copy_id INT,
    delivery_method_id INT,
    requested_at DATETIME,
    status VARCHAR(20),
    FOREIGN KEY (borrower_user_id) REFERENCES `user`(user_id),
    FOREIGN KEY (copy_id) REFERENCES `bookcopy`(copy_id),
    FOREIGN KEY (delivery_method_id) REFERENCES `deliverymethod`(delivery_method_id)
);
```

Mit dieser Abfrage wird geprüft, ob Reservierungen korrekt mit Benutzer, Buch und Liefermethode verknüpft sind.

```
SELECT r.reservation_id, u.full_name AS borrower, b.title, dm.method_name, r.status,
r.requested_at
FROM `reservation` r
JOIN `user` u ON r.borrower_user_id = u.user_id
JOIN `bookcopy` bc ON r.copy_id = bc.copy_id
JOIN `book` b ON bc.book_id = b.book_id
JOIN `deliverymethod` dm ON r.delivery_method_id = dm.delivery_method_id
ORDER BY r.requested_at DESC;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `reservation` (borrower_user_id, copy_id,
delivery_method_id, requested_at, status) VALUES
(2,1,1,NOW(),'offen'),
(3,2,1,NOW(),'offen'),
(4,3,2,NOW(),'angenommen'),
(5,4,1,NOW(),'offen'),
(6,5,2,NOW(),'abgelehnt'),
(7,6,1,NOW(),'offen'),
(8,7,2,NOW(),'angenommen'),
(9,8,1,NOW(),'offen'),
(10,9,2,NOW(),'offen'),
(1,10,1,NOW(),'angenommen');
```

Ergebnis der Abfrage:

reservation_id	borrower	title	method_name	status	requested_at 1
1	Anna Schmidt	Buch 1	PICKUP	offen	2026-03-27 17:37:03
9	Lea Mayer	Buch 9	SHIPPING	offen	2026-03-27 17:37:03
8	Tim Braun	Buch 8	PICKUP	offen	2026-03-27 17:37:03
7	Mia Koch	Buch 7	SHIPPING	angenommen	2026-03-27 17:37:03
6	Jan Wolf	Buch 6	PICKUP	offen	2026-03-27 17:37:03
5	Sara Klein	Buch 5	SHIPPING	abgelehnt	2026-03-27 17:37:03
4	Paul Fischer	Buch 4	PICKUP	offen	2026-03-27 17:37:03
3	Lisa Weber	Buch 3	SHIPPING	angenommen	2026-03-27 17:37:03
2	Tom Becker	Buch 2	PICKUP	offen	2026-03-27 17:37:03
10	Max Mustermann	Buch 10	PICKUP	angenommen	2026-03-27 17:37:03

Loan

Dokumentiert abgeschlossene Ausleihvorgänge mit Start-, Fälligkeits- und Rückgabedatum.

Zur Erstellung der Tabelle:

```
CREATE TABLE `loan` (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  borrower_user_id INT,
  copy_id INT,
  location_id INT,
  start_date DATE,
  due_date DATE,
  return_date DATE,
  FOREIGN KEY (borrower_user_id) REFERENCES `user`(user_id),
  FOREIGN KEY (copy_id) REFERENCES `bookcopy`(copy_id),
  FOREIGN KEY (location_id) REFERENCES `pickuplocation`(location_id)
);
```

Mit dieser Abfrage wird geprüft, ob Ausleihvorgänge korrekt gespeichert und nachvollziehbar sind.

```
SELECT l.loan_id, u.full_name AS borrower, b.title, l.start_date, l.due_date, l.return_date
FROM `loan` l
JOIN `user` u ON l.borrower_user_id = u.user_id
JOIN `bookcopy` bc ON l.copy_id = bc.copy_id
JOIN `book` b ON bc.book_id = b.book_id
ORDER BY l.start_date DESC;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `loan` (borrower_user_id, copy_id, location_id,
start_date, due_date, return_date) VALUES
(2,1,1,'2025-01-01','2025-01-14',NULL),
(3,2,2,'2025-01-02','2025-01-12',NULL),
(4,3,3,'2025-01-03','2025-01-10',NULL),
(5,4,4,'2025-01-04','2025-01-18',NULL),
(6,5,5,'2025-01-05','2025-01-09',NULL),
(7,6,6,'2025-01-06','2025-01-16',NULL),
(8,7,7,'2025-01-07','2025-01-19',NULL),
(9,8,8,'2025-01-08','2025-01-15',NULL),
(10,9,9,'2025-01-09','2025-01-20',NULL),
(1,10,10,'2025-01-10','2025-01-30',NULL);
```

Ergebnis der Abfrage:

loan_id	borrower	title	start_date 1	due_date	return_date
10	Max Mustermann	Buch 10	2025-01-10	2025-01-30	NULL
9	Lea Mayer	Buch 9	2025-01-09	2025-01-20	NULL
8	Tim Braun	Buch 8	2025-01-08	2025-01-15	NULL
7	Mia Koch	Buch 7	2025-01-07	2025-01-19	NULL
6	Jan Wolf	Buch 6	2025-01-06	2025-01-16	NULL
5	Sara Klein	Buch 5	2025-01-05	2025-01-09	NULL
4	Paul Fischer	Buch 4	2025-01-04	2025-01-18	NULL
3	Lisa Weber	Buch 3	2025-01-03	2025-01-10	NULL
2	Tom Becker	Buch 2	2025-01-02	2025-01-12	NULL
1	Anna Schmidt	Buch 1	2025-01-01	2025-01-14	NULL

Review

Erfasst Bewertungen und Kommentare von Benutzenden zu ausgeliehenen Büchern.

Zur Erstellung der Tabelle:

```
CREATE TABLE `review` (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    reviewer_user_id INT,
    copy_id INT,
    rating TINYINT,
    comment TEXT,
    FOREIGN KEY (reviewer_user_id) REFERENCES `user`(user_id),
    FOREIGN KEY (copy_id) REFERENCES `bookcopy`(copy_id)
);
```

Mit dieser Abfrage wird geprüft, ob Bewertungen korrekt mit Benutzenden und Buch-Exemplaren verknüpft sind.

```
SELECT rv.review_id, u.full_name AS reviewer, b.title, rv.rating, rv.comment
FROM `review` rv
JOIN `user` u ON rv.reviewer_user_id = u.user_id
JOIN `bookcopy` bc ON rv.copy_id = bc.copy_id
JOIN `book` b ON bc.book_id = b.book_id
ORDER BY rv.rating DESC, rv.review_id;
```

Zum Befüllen der Tabelle:

```
INSERT INTO `review` (reviewer_user_id, copy_id, rating, comment) VALUES
(2,1,5,'Super'),
(3,2,4,'Gut'),
(4,3,3,'Okay'),
(5,4,5,'Top'),
(6,5,2,'Schlecht'),
(7,6,4,'Gut'),
(8,7,5,'Mega'),
(9,8,3,'Mittel'),
(10,9,4,'Gut'),
(1,10,5,'Perfekt');
```

Ergebnis der Abfrage:

review_id	reviewer	title	rating ▼ 1	comment
1	Anna Schmidt	Buch 1	5	Super
4	Paul Fischer	Buch 4	5	Top
7	Mia Koch	Buch 7	5	Mega
10	Max Mustermann	Buch 10	5	Perfekt
2	Tom Becker	Buch 2	4	Gut
6	Jan Wolf	Buch 6	4	Gut
9	Lea Mayer	Buch 9	4	Gut
3	Lisa Weber	Buch 3	3	Okay
8	Tim Braun	Buch 8	3	Mittel
5	Sara Klein	Buch 5	2	Schlecht

Komplexere SQL-Abfragen

Diese Abfrage dient der Analyse der Nachfrage. Sie zeigt, welche Buchtitel besonders häufig reserviert werden und unterstützt damit die Auswertung der Beliebtheit einzelner Titel.

```
SELECT b.title,
    ROUND(AVG(rv.rating), 2) AS durchschnittsbewertung,
    COUNT(rv.review_id) AS anzahl_bewertungen
FROM review rv
JOIN bookcopy bc ON rv.copy_id = bc.copy_id
JOIN book b ON bc.book_id = b.book_id
GROUP BY b.book_id, b.title
ORDER BY durchschnittsbewertung DESC, anzahl_bewertungen DESC;
```

title	durchschnittsbewertung 1	anzahl_bewertungen 2
Buch 10	5.00	1
Buch 7	5.00	1
Buch 4	5.00	1
Buch 1	5.00	1
Buch 9	4.00	1
Buch 2	4.00	1
Buch 6	4.00	1
Buch 3	3.00	1
Buch 8	3.00	1
Buch 5	2.00	1

Mit dieser Abfrage wird ausgewertet, welche Benutzenden aktuell wie viele Bücher ausgeliehen haben. Das ist für Monitoring und Verwaltungszwecke relevant.

```
SELECT u.full_name,
    COUNT(l.loan_id) AS aktive_ausleihen
FROM user u
LEFT JOIN loan l
  ON u.user_id = l.borrower_user_id
 AND l.return_date IS NULL
GROUP BY u.user_id, u.full_name
ORDER BY aktive_ausleihen DESC, u.full_name;
```

full_name	aktive_ausleihen 1
Anna Schmidt	1
Jan Wolf	1
Lea Mayer	1
Lisa Weber	1
Max Mustermann	1
Mia Koch	1
Paul Fischer	1
Sara Klein	1
Tim Braun	1
Tom Becker	1

Komplexere SQL-Abfragen 2

Diese Auswertung zeigt, welche Liefermethode bevorzugt wird. Damit wird der funktionale Nutzen der Tabelle DeliveryMethod deutlicher.

```
SELECT dm.method_name,
    COUNT(r.reservation_id) AS anzahl_nutzung
FROM reservation r
JOIN deliverymethod dm ON r.delivery_method_id = dm.delivery_method_id
GROUP BY dm.delivery_method_id, dm.method_name
ORDER BY anzahl_nutzung DESC;
```

method_name	anzahl_nutzung 1
PICKUP	6
SHIPPING	4

Diese Abfrage bildet einen praxisnahen Geschäftsfall ab: offene und bereits überfällige Ausleihen können so gezielt identifiziert werden.

```
SELECT u.full_name AS borrower,
    b.title,
    l.start_date,
    l.due_date
FROM loan l
JOIN user u ON l.borrower_user_id = u.user_id
JOIN bookcopy bc ON l.copy_id = bc.copy_id
JOIN book b ON bc.book_id = b.book_id
WHERE l.return_date IS NULL
  AND l.due_date < CURDATE()
ORDER BY l.due_date;
```

borrower	title	start_date	due_date 1
Sara Klein	Buch 5	2025-01-05	2025-01-09
Lisa Weber	Buch 3	2025-01-03	2025-01-10
Tom Becker	Buch 2	2025-01-02	2025-01-12
Anna Schmidt	Buch 1	2025-01-01	2025-01-14
Tim Braun	Buch 8	2025-01-08	2025-01-15
Jan Wolf	Buch 6	2025-01-06	2025-01-16
Paul Fischer	Buch 4	2025-01-04	2025-01-18
Mia Koch	Buch 7	2025-01-07	2025-01-19
Lea Mayer	Buch 9	2025-01-09	2025-01-20
Max Mustermann	Buch 10	2025-01-10	2025-01-30

View zur Vereinfachung häufiger Suchabfragen

Die View bündelt häufig benötigte Informationen zu verfügbaren Buchexemplaren in einer wiederverwendbaren Sicht. Dadurch werden Suchabfragen vereinfacht und die Lesbarkeit der SQL-Abfragen verbessert.

```
CREATE VIEW view_available_books AS
SELECT bc.copy_id,
    b.title,
    u.full_name AS owner,
    pl.city,
    bc.condition_grade,
    bc.deposit_amount,
    bc.status
FROM bookcopy bc
JOIN book b ON bc.book_id = b.book_id
JOIN user u ON bc.owner_user_id = u.user_id
JOIN copyavailability ca ON bc.copy_id = ca.copy_id
JOIN pickuplocation pl ON ca.location_id = pl.location_id
WHERE bc.status = 'verfügbar';
```

```
SELECT * FROM `view_available_books`
```

Messen [Inline bearbeiten] [Bearbeiten] [SQL erklären] [PHP-Code erzeugen] [Aktualisieren]

Alles anzeigen | Anzahl der Datensätze: 25 Zeilen filtern: Diese Tabelle durchsuchen

Zusätzliche Optionen

	copy_id	title	owner	city	condition_grade	deposit_amount	status
Bearbeiten Kopieren Löschen	1	Buch 1	Max Mustermann	Karlsruhe	gut	5.00	verfügbar
Bearbeiten Kopieren Löschen	2	Buch 2	Anna Schmidt	Karlsruhe	neu	3.00	verfügbar
Bearbeiten Kopieren Löschen	3	Buch 3	Tom Becker	Karlsruhe	mittel	2.00	verfügbar
Bearbeiten Kopieren Löschen	4	Buch 4	Lisa Weber	Karlsruhe	gut	4.00	verfügbar
Bearbeiten Kopieren Löschen	5	Buch 5	Paul Fischer	Karlsruhe	alt	1.00	verfügbar
Bearbeiten Kopieren Löschen	6	Buch 6	Sara Klein	Karlsruhe	gut	2.00	verfügbar
Bearbeiten Kopieren Löschen	7	Buch 7	Jan Wolf	Karlsruhe	neu	3.00	verfügbar
Bearbeiten Kopieren Löschen	8	Buch 8	Mia Koch	Karlsruhe	mittel	2.00	verfügbar
Bearbeiten Kopieren Löschen	9	Buch 9	Tim Braun	Karlsruhe	gut	5.00	verfügbar
Bearbeiten Kopieren Löschen	10	Buch 10	Lea Mayer	Karlsruhe	neu	6.00	verfügbar

Beispielabfrage zur View:

```
SELECT *
FROM view_available_books
ORDER BY city, title;
```

	copy_id	title 2	owner	city 1	condition_grade	deposit_amount	status
Bearbeiten Kopieren Löschen	1	Buch 1	Max Mustermann	Karlsruhe	gut	5.00	verfügbar
Bearbeiten Kopieren Löschen	10	Buch 10	Lea Mayer	Karlsruhe	neu	6.00	verfügbar
Bearbeiten Kopieren Löschen	2	Buch 2	Anna Schmidt	Karlsruhe	neu	3.00	verfügbar
Bearbeiten Kopieren Löschen	3	Buch 3	Tom Becker	Karlsruhe	mittel	2.00	verfügbar
Bearbeiten Kopieren Löschen	4	Buch 4	Lisa Weber	Karlsruhe	gut	4.00	verfügbar
Bearbeiten Kopieren Löschen	5	Buch 5	Paul Fischer	Karlsruhe	alt	1.00	verfügbar
Bearbeiten Kopieren Löschen	6	Buch 6	Sara Klein	Karlsruhe	gut	2.00	verfügbar
Bearbeiten Kopieren Löschen	7	Buch 7	Jan Wolf	Karlsruhe	neu	3.00	verfügbar
Bearbeiten Kopieren Löschen	8	Buch 8	Mia Koch	Karlsruhe	mittel	2.00	verfügbar
Bearbeiten Kopieren Löschen	9	Buch 9	Tim Braun	Karlsruhe	gut	5.00	verfügbar

Trigger zur automatischen Statuspflege

Der Trigger bildet typische Geschäftslogik direkt in der Datenbank ab. Sobald ein Ausleihvorgang erzeugt wird, wird der Status des Exemplars automatisch aktualisiert. So bleibt der Datenbestand konsistent.

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_loan
AFTER INSERT ON loan
FOR EACH ROW
BEGIN
   UPDATE bookcopy
   SET status = 'ausgeliehen'
   WHERE copy_id = NEW.copy_id;
END$$
DELIMITER ;
```

Dieser Trigger sorgt dafür, dass ein Exemplar nach Eintragung der Rückgabe wieder automatisch als verfügbar geführt wird. Damit wird die Rückgabe fachlich korrekt im System abgebildet.

```
DELIMITER $$
CREATE TRIGGER trg_after_update_loan_return
AFTER UPDATE ON loan
FOR EACH ROW
BEGIN
   IF NEW.return_date IS NOT NULL THEN
      UPDATE bookcopy
      SET status = 'verfügbar'
      WHERE copy_id = NEW.copy_id;
   END IF;
END$$
DELIMITER ;
```